

Exercise #3 - Threading/Tasking and Real-Time Synchronization

Mason McGaffin

Exercise #3

ECEN 5623 Real-Time Embedded Systems

University of Colorado Boulder

June 24, 2026

Contents

1 PRIORITY INHERENCE PROTOCOLS	3
1.1 SHA, RAJKUMAR, ET AL	3
1.2 SIEWERT	4
1.3 LINUX COMMUNITY RESISTANCE	4
1.4 PRIORITY CEILING EMULATION VS. PRIORITY INHERITANCE	5
2 THREAD SAFE FUNCTIONS	6
2.1 TERMINOLOGY.....	6
2.2 RE-ENTRANT METHODS AND IMPACTS	7
2.3 SAFELY UPDATING A COMPLEX STATE.....	8
3 EXAMPLE-SYNC-UPDATED-2	11
3.1 ROOT CAUSE ANALYSIS	11
3.1.1 <i>deadlock and deadlock_timeout</i>	11
3.1.2 <i>pthread3, pthread3amp, pthread3ok</i>	13
3.2 FIXES & KERNEL PATCH.....	14
4 POSIX MESSAGE QUEUES	16
4.1 IMPLEMENTING RT-LINUX PTHREADS – FIFO	16
4.2 HEAP VS SIMPLE MESSAGE QUEUES	17
4.3 UNBOUNDED INVERSION	19
5 LINUX WATCHDOG DAEMON.....	20
5.1 USING WATCHDOG IN COMPLEX STATE UPDATE	20
6 REFERENCES.....	21
7 APPENDIX	22

Note. For the RTES course, I, Mason McGaffin, will be using the Jetson Orin Nano as my embedded system of choice. Although this system is over resourced for our application, it is convenient to use given my current development situation.

1 Priority Inherence Protocols

Read Sha, Rajkumar, et al paper, "Priority Inheritance Protocols: An Approach to Real-Time Synchronization".

a) Summarize 3 main key points the paper makes. Read my summary paper on the topic as well, which might be easier to understand.

b) Read the historical positions of Linux Torvalds as described by Jonathan Corbet and Ingo Molnar and Thomas Gleixner on this topic as well. Note that more recently Red Hat has added support for priority ceiling emulation and priority inheritance and has a great overview on use of these POSIX real-time extension features [here](#) – general real-time overview. The key systems calls are `pthread_mutexattr_setprotocol`, `pthread_mutexattr_getprotocol` as well as `pthread_mutex_setpriorityceiling` and `pthread_mutex_getpriorityceiling`. Why did some of the Linux community resist addition of priority ceiling and/or priority inheritance until more recently (Linux has been distributed since the early 1990's and the problem and solutions were known before this).

c) Given that there are two solutions to unbounded priority inversion, priority ceiling emulation and priority inheritance, based upon your reading on support for both by POSIX real-time extensions, which do you think is best to use and why?

1.1 Sha, Rajkumar, et al

1. First, the paper makes a key point to highlight the severity of uncontrolled priority inversion. The paper defines what priority inversion is – a high-priority task blocked by a lower-priority task that holds a shared resource – and then addresses the impact of this critical flaw presented by synchronization mechanism such as mutexes and semaphores. The authors introduce two protocols to mitigate the risk of priority inversion.
2. The second key point the paper makes is the Basic Priority Inheritance Protocol. The authors present this protocol as a minimal and simple solution to the flaw and explain its limitations. The protocol mechanism works by elevating a low-priority task to the priority of the highest-priority task it currently blocks. The temporary inheritance of the higher priority prevents intermediate tasks from preempting the task (originally low-priority) holding the shared resource, allowing it to release the resource, unblock the higher-priority task, and return to its original priority. The limitations of the protocol include that there is no deadlock prevention and significant latency in the case of “chained blocking,” during which a task needs multiple shared resource that are sequentially blocked by lower-priority tasks.
3. The third key point of the paper is that the Priority Ceiling Protocol, while more complex, provides a better solution to the priority inversion problem that also addresses the

limitations of the Basic Priority Inheritance Protocol. The protocol utilizes a priority ceiling, assigned to each semaphore, equal to the highest priority task that could ever use it. Next, a task, call it 'T1', can only enter a critical section if its priority is higher than the priority ceilings of all semaphores currently locked. If not, T1 is suspended and the task holding the conflicting lock (higher ceiling) inherits the priority of T1. This prevents deadlocks entirely and eliminates chain blocking. The authors also highlight that this protocol improves worst-case predictability because a high-priority task can only be blocked for a single critical section of a low-priority task.

1.2 Siewert

The article discusses much more than priority inversion, discussing SoC trade-offs, virtual memory, and cache locking, but the three main key points with regards to priority inversion are as follows.

1. The first main point that Siewert makes is the triggering conditions and impact of priority inversion. He claims that CPU multiplexing, shared resources using mutual exclusion, **and** a middle-priority service must exist to cause a priority inversion event. Siewert discusses that priority inversion is a severe threat to system uptime. When uncontrolled, these events typically cause deadlock which will cause a system to lock up, typically forcing a recovery via a hardware watchdog timer and a system reboot. Siewert discusses the impact this can have on high-availability and mission critical systems.
2. The next main point the author makes is that three protocols allow for one of the triggering conditions to be avoided which prevents an uncontrolled priority inversion. These protocols are Priority Inheritance Protocol, which is described similar to the Basic Priority Inheritance Protocol referenced in Section 1.1, Highest Locker, and Priority Ceiling Protocol, also referenced in Section 1.1.
3. Lastly, Siewert discusses the historical push-back from the embedded Linux community regarding priority inversion management. Commercial RTOS offer flags to enable inversion-safe semaphores. In contrast, standard Linux has lacked POSIX compliance for priority ceilings and inheritance. Siewert comments that system architects must have deep kernel expertise as they will need to secure the system against priority inversion manually.

1.3 Linux Community Resistance

Linus Torvalds' quote "Friends don't let friends use priority inheritance ... If you really need it, your system is broken anyway" [6] sums up how many in the Linux community felt about priority inheritance in the kernel. With this animosity towards the subject, many developers scraped systems that ran into these problems and went back to the drawing board.

The main philosophy behind this resistance was to prioritize Linux as a general-purpose operating system, not a strict RTOS. Because it is not an RTOS, Torvalds argued that if an

uncontrolled priority inversion caused your system to crash, then it was an underlying system design flaw. Core developers feared that the complexity of implementing priority inversion solutions would create new dangerous deadlocks and race conditions, and hide deadlocks under the hood.

Historically, this position of resistance makes a lot of sense. Priority inversion solutions were very complex. Linux was positioned as a general purpose operating system and not an RTOS, thus validating that resulting system crashes lied in the system design. It also supported his fear of compromising the entire architecture of the OS for a small portion of the community. This presented a major hurdle for embedded and real-time Linux developers. Problems were growing more and more complex and this resistance resulted in the development of proprietary alternatives which delayed the adoption of Linux in the embedded market.

1.4 Priority Ceiling Emulation vs. Priority Inheritance

Based on the sources provided and additional research on the two policies, I confidently believe that priority ceiling emulation is a much better approach to the problem of uncontrolled priority inversion. Granted, this approach requires a much more extensive system design and will inevitably result in some strenuous runtime debugging. However, when implemented correctly, the benefits that this approach provides significantly outweigh the development cost in most scenarios. Priority ceiling emulation creates more deterministic systems, prevents deadlock, and eliminates chained blocking. This is absolutely critical for real-time systems that have strict timing requirements and require high availability and low latency. Therefore, priority ceiling emulation is the more complex, but better solution for unbounded priority inversion in real-time embedded systems.

2 Thread Safe Functions

Review the terminology guide (glossary in the textbook).

a) Describe clearly what it means to write "thread safe" functions that are "re-entrant". There are generally three main ways to do this:

- 1) pure functions that use only stack and have no global memory*
- 2) functions which use thread indexed global data*
- 3) functions which use shared memory global data, but synchronize access to it using a MUTEX semaphore critical section wrapper.*

b) Describe each method and how you would code it and how it would impact real-time threads/tasks. (E.g., if you were to create a simple function to average 2 numbers and return the average, how would you make sure this function is thread safe using each of the three methods).

c) Now, using a MUTEX, provide an example using RT-Linux Pthreads that does a thread safe update of a complex state (3 or more numbers – e.g., Latitude, Longitude and Altitude of a location) with a timestamp (pthread_mutex_lock). Your code should include two threads and one should update a timespec structure contained in a structure that includes a double precision position and attitude state of {Lat, Long, Altitude and Roll, Pitch, Yaw at Sample_Time} and the other should read it and never disagree on the values as function of time. You can just make up values for the navigational state using math library function generators (e.g., use simple periodic functions for Roll, Pitch, Yaw $\sin(x)$, $\cos(x^2)$, and $\cos(x)$, where $x=\text{time}$ and linear functions for Lat, Long, Alt) and see http://linux.die.net/man/3/clock_gettime for how to add a precision timestamp. The second thread should read the times-stamped state without the possibility of data corruption (partial update of one of the 6 floating point values). There should be no disagreement between the functions and the state reader for any point in time. Run this for 180 seconds with a 1 Hz update rate and a 0.1 Hz read rate. Make sure the 18 values read are correct.

2.1 Terminology

The one-way implication, "All re-entrant functions are thread-safe, but not all thread-safe functions are re-entrant," summarizes the relationship between these two types of functions. Thread-safe functions can be called by multiple threads at the same time without any undefined or dangerous behavior. These functions can be called "re-entrant" if its execution can safely be interrupted in the middle of its execution and called (from the start) by a different caller; the original execution must safely resume from the exact point of interruption. Re-entrant functions have the following characteristics and limitations.

- No global or static variables
- Must not call other non-reentrant functions (i.e. malloc)
- Must only rely on stack allocated arguments and variables
- Any shared resources must be safely and synchronously accessed – no locks

2.2 Re-entrant Methods and Impacts

1. *pure functions that use only stack and have no global memory*

This method is the simplest and easiest to implement. It dependent entirely on the input arguments and local variables allocated on the stack. Each thread uses its own stack. No variables are shared. There is no chance of data-race and the function is thread-safe and re-entrant. This method is deterministic. There is no risk of priority inversion or deadlock and it is interrupt safe.

```
double average(double a, double b) {  
    double result = (a + b) / 2.0;  
  
    return result;  
}
```

Example 2.1

2. *functions which use thread indexed global data*

This method utilized global data that is specific to each thread. Each thread has its own slot in the global space and the data remains isolated. This makes the function thread-safe. This execution has no blocking or locking so that execution is deterministic. There is no risk of priority inversion or deadlock. Also, the function has one case that makes it not strictly re-entrant. If an interrupt occurs during execution on a thread right before the return statement and the ISR calls this function, with the same thread ID, the global variable will be overwritten with the ISR parameters passed.

```
#include <pthread.h>  
  
__thread double thread_average = 0.0;  
  
double average(double a, double b) {  
    thread_average = (a + b) / 2.0;  
  
    return thread_average;  
}
```

Example 2.2

3. *functions which use shared memory global data, but synchronize access to it using a MUTEX semaphore critical section wrapper.*

This method uses a MUTEX to create a critical section to block other threads from accessing shared data at the same time. This ensures that the function is thread-safe, as other threads will have to wait for the current thread holding the mutex to release it first, but it introduces the threat of uncontrolled priority inversion and makes the function non-reentrant. If an interrupt fires

when a thread holds the mutex and the ISR calls the function, the handler will block on the MUTEX forever causing deadlock. The MUTEX is a blocking mechanism that makes the function execution time non-deterministic. It has significant impact on real-time applications that have strict deadlines. Blocking also introduces a risk of priority inversion for systems that have three (3) or more threads. In order to prevent this, the mutex must be initialized with an inversion safe protocol, such as PTHREAD_PRIO_INHERIT.

```
#include <pthread.h>

double global_average = 0.0;
pthread_mutex_t avg_mutex; //initialized elsewhere

double average(double a, double b) {
    // BEGIN CRITICAL SECTION
    pthread_mutex_lock(&avg_mutex);

    double result = (a + b) / 2.0;
    global_average = result;

    // END CRITICAL SECTION
    pthread_mutex_unlock(&avg_mutex);

    return result;
}
```

Example 2.3

2.3 Safely Updating a Complex State

This [commit](#) contains the code developed for the update_complex_state application:

The application uses two threads, a writer and reader thread, that access a shared state resource. This shared resource is protected by a mutex associated with the state. The mutex is a member of the state_t struct; this is to reinforce the principle that mutexes are associated with data, not code. Currently, the application uses C macros to configure the duration, update rate, and read rate. As future work, these variables could be passed as arguments during the application execution command. Based on the update/read rate the respective thread sleeps until it is required to do its respective task. Every update by the writer thread and read by the reader thread is logged using syslog, as shown in Figure 2.4. The log denotes the thread name and the corresponding position and attitude state and is timestamped using CLOCK_REALTIME in seconds.

```

Jun 23 10:33:34 jetson-orin-nano update_complex_state 3000472: starting complex state update - Duration: 180, Update rate: 1.00, Read rate: 0.10
Jun 23 10:33:34 jetson-orin-nano update_complex_state 3000472: [1782232414, 3919] WRITER (CPU 4) - lat=0.000001 lon=0.000028 alt=0.000035 roll=0.000138 pitch=1.000000 yaw=1.000000
Jun 23 10:33:35 jetson-orin-nano update_complex_state 3000472: [1782232415, 3922] WRITER (CPU 4) - lat=0.010004 lon=0.200071 alt=0.250088 roll=0.841661 pitch=0.539709 yaw=0.540006
Jun 23 10:33:36 jetson-orin-nano update_complex_state 3000472: [1782232416, 3928] WRITER (CPU 4) - lat=0.020010 lon=0.400200 alt=0.500250 roll=0.908800 pitch=0.650508 yaw=0.417057
Jun 23 10:33:37 jetson-orin-nano update_complex_state 3000472: [1782232417, 3932] WRITER (CPU 4) - lat=0.030014 lon=0.600274 alt=0.750342 roll=0.139765 pitch=0.914484 yaw=0.990185
Jun 23 10:33:38 jetson-orin-nano update_complex_state 3000472: [1782232418, 3934] WRITER (CPU 4) - lat=0.040016 lon=0.800327 alt=1.000409 roll=0.757871 pitch=0.953807 yaw=0.652404
Jun 23 10:33:39 jetson-orin-nano update_complex_state 3000472: [1782232419, 3939] WRITER (CPU 4) - lat=0.050021 lon=1.000414 alt=1.250518 roll=0.958334 pitch=0.993733 yaw=0.285649
Jun 23 10:33:40 jetson-orin-nano update_complex_state 3000472: [1782232420, 3943] WRITER (CPU 4) - lat=0.060025 lon=1.200498 alt=1.500622 roll=0.277025 pitch=0.098290 yaw=0.960863
Jun 23 10:33:41 jetson-orin-nano update_complex_state 3000472: [1782232421, 3947] WRITER (CPU 4) - lat=0.070029 lon=1.400572 alt=1.750715 roll=0.659140 pitch=0.338538 yaw=0.752020
Jun 23 10:33:42 jetson-orin-nano update_complex_state 3000472: [1782232422, 3951] WRITER (CPU 4) - lat=0.080033 lon=1.600653 alt=2.000816 roll=0.988878 pitch=0.343270 yaw=0.148730
Jun 23 10:33:43 jetson-orin-nano update_complex_state 3000472: [1782232423, 3954] WRITER (CPU 4) - lat=0.090036 lon=1.800727 alt=2.250909 roll=0.408804 pitch=0.816210 yaw=0.912622
Jun 23 10:33:44 jetson-orin-nano update_complex_state 3000472: [1782232423, 3954] READER (CPU 3) - lat=0.090036 lon=1.800727 alt=2.250909 roll=0.408804 pitch=0.816210 yaw=0.912622
Jun 23 10:33:44 jetson-orin-nano update_complex_state 3000472: [1782232424, 3958] WRITER (CPU 4) - lat=0.100040 lon=2.000798 alt=2.500998 roll=0.547367 pitch=0.899967 yaw=0.836893
Jun 23 10:33:45 jetson-orin-nano update_complex_state 3000472: [1782232425, 3961] WRITER (CPU 4) - lat=0.110042 lon=2.200848 alt=2.751061 roll=0.999962 pitch=0.141552 yaw=0.008668
Jun 23 10:33:46 jetson-orin-nano update_complex_state 3000472: [1782232426, 3964] WRITER (CPU 4) - lat=0.120046 lon=2.400915 alt=3.001144 roll=0.532707 pitch=0.919711 yaw=0.846300
Jun 23 10:33:47 jetson-orin-nano update_complex_state 3000472: [1782232427, 3967] WRITER (CPU 4) - lat=0.130049 lon=2.600986 alt=3.251232 roll=0.424633 pitch=0.868880 yaw=0.905365
Jun 23 10:33:48 jetson-orin-nano update_complex_state 3000472: [1782232428, 3975] WRITER (CPU 4) - lat=0.140057 lon=2.801138 alt=3.501422 roll=0.991369 pitch=0.189081 yaw=0.131100
Jun 23 10:33:49 jetson-orin-nano update_complex_state 3000472: [1782232429, 3979] WRITER (CPU 4) - lat=0.150061 lon=3.001211 alt=3.751513 roll=0.645677 pitch=0.529304 yaw=0.763611
Jun 23 10:33:50 jetson-orin-nano update_complex_state 3000472: [1782232430, 3986] WRITER (CPU 4) - lat=0.160068 lon=3.201362 alt=4.001702 roll=0.294417 pitch=0.177170 yaw=0.955677
Jun 23 10:33:51 jetson-orin-nano update_complex_state 3000472: [1782232431, 3990] WRITER (CPU 4) - lat=0.170072 lon=3.401434 alt=4.251793 roll=0.963346 pitch=0.976470 yaw=0.268262
Jun 23 10:33:52 jetson-orin-nano update_complex_state 3000472: [1782232432, 3998] WRITER (CPU 4) - lat=0.180080 lon=3.601590 alt=4.501988 roll=0.745713 pitch=0.763359 yaw=0.666268
Jun 23 10:33:53 jetson-orin-nano update_complex_state 3000472: [1782232433, 4001] WRITER (CPU 4) - lat=0.190083 lon=3.801664 alt=4.752080 roll=0.158098 pitch=0.999453 yaw=0.987423
Jun 23 10:33:54 jetson-orin-nano update_complex_state 3000472: [1782232433, 4001] READER (CPU 3) - lat=0.190083 lon=3.801664 alt=4.752080 roll=0.158098 pitch=0.999453 yaw=0.987423
Jun 23 10:33:54 jetson-orin-nano update_complex_state 3000472: [1782232434, 4008] WRITER (CPU 4) - lat=0.200097 lon=4.001735 alt=5.002169 roll=0.916451 pitch=0.204512 yaw=0.400146
Jun 23 10:33:55 jetson-orin-nano update_complex_state 3000472: [1782232435, 4007] WRITER (CPU 4) - lat=0.210089 lon=4.201787 alt=5.252233 roll=0.831729 pitch=0.018468 yaw=0.555182
Jun 23 10:33:56 jetson-orin-nano update_complex_state 3000472: [1782232436, 4015] WRITER (CPU 4) - lat=0.220097 lon=4.401934 alt=5.502417 roll=0.018520 pitch=0.813713 yaw=0.999828
Jun 23 10:33:57 jetson-orin-nano update_complex_state 3000472: [1782232437, 4018] WRITER (CPU 4) - lat=0.230100 lon=4.602007 alt=5.752508 roll=0.851523 pitch=0.103057 yaw=0.524317
Jun 23 10:33:58 jetson-orin-nano update_complex_state 3000472: [1782232438, 4026] WRITER (CPU 4) - lat=0.240108 lon=4.802157 alt=6.002696 roll=0.900951 pitch=0.035519 yaw=0.433920
Jun 23 10:33:59 jetson-orin-nano update_complex_state 3000472: [1782232439, 4029] WRITER (CPU 4) - lat=0.250111 lon=5.002228 alt=6.252785 roll=0.121301 pitch=0.928580 yaw=0.992616
Jun 23 10:34:00 jetson-orin-nano update_complex_state 3000472: [1782232440, 4033] WRITER (CPU 4) - lat=0.260115 lon=5.202305 alt=6.502881 roll=0.769962 pitch=0.402149 yaw=0.638089
Jun 23 10:34:01 jetson-orin-nano update_complex_state 3000472: [1782232441, 4038] WRITER (CPU 4) - lat=0.270120 lon=5.402392 alt=6.752990 roll=0.952813 pitch=0.699193 yaw=0.303557
Jun 23 10:34:02 jetson-orin-nano update_complex_state 3000472: [1782232442, 4046] WRITER (CPU 4) - lat=0.280128 lon=5.602552 alt=7.003190 roll=0.258603 pitch=0.775343 yaw=0.965984
Jun 23 10:34:03 jetson-orin-nano update_complex_state 3000472: [1782232443, 4050] WRITER (CPU 4) - lat=0.290132 lon=5.802632 alt=7.253290 roll=0.673422 pitch=0.983248 yaw=0.739259
Jun 23 10:34:04 jetson-orin-nano update_complex_state 3000472: [1782232443, 4050] READER (CPU 3) - lat=0.290132 lon=5.802632 alt=7.253290 roll=0.673422 pitch=0.983248 yaw=0.739259
Jun 23 10:34:04 jetson-orin-nano update_complex_state 3000472: [1782232444, 4053] WRITER (CPU 4) - lat=0.300135 lon=6.002706 alt=7.503383 roll=0.985854 pitch=0.785520 yaw=0.167606
Jun 23 10:34:05 jetson-orin-nano update_complex_state 3000472: [1782232445, 4056] WRITER (CPU 4) - lat=0.310138 lon=6.202756 alt=7.753445 roll=0.391394 pitch=0.864217 yaw=0.920223
Jun 23 10:34:06 jetson-orin-nano update_complex_state 3000472: [1782232446, 4063] WRITER (CPU 4) - lat=0.320145 lon=6.402904 alt=8.003630 roll=0.963481 pitch=0.717736 yaw=0.826129

```

Figure 2.4 Syslog output for update_complex_state

The application has a functional test framework that executes when the argument ‘-t’, for testing mode, is passed with application execution command. The testing framework completes the standard application execution but saves the necessary states to ensure that the writer and reader thread agree. In this specified example, eighteen (18) states are stored, for both writer and reader. With a duration of 180 seconds and a read rate of 0.1 Hz, the reader thread logs the state eighteen (18) times. This corresponds with every tenth update made by the writer thread ($1\text{Hz} / 0.1\text{ Hz} = 10$). Therefore, every tenth update made by the writer is stored, totaling eighteen ($180 / 10 = 18$) entries. At the end of the application execution, the stored states are compared. The testing framework uses the fabs() API from the math.h library and a small epsilon value ($1\text{e-}9$) in order to more accurately compare the double type state members. If any of the members do not agree, the test fails and is reported. If all of the members agree, the test is successful, as seen in Figure 2.5.

```
mason@jetson-orin-nano:~/RTES_5623/ex3$ ./update_complex_state -t
Logging to syslog with identifier 'update_complex_state'
Beginning update_complex_state test - Duration: 180, Update rate: 1.00, Read rate: 0.10
[SUCCESS] TEST 0 - All 6 members match
[SUCCESS] TEST 1 - All 6 members match
[SUCCESS] TEST 2 - All 6 members match
[SUCCESS] TEST 3 - All 6 members match
[SUCCESS] TEST 4 - All 6 members match
[SUCCESS] TEST 5 - All 6 members match
[SUCCESS] TEST 6 - All 6 members match
[SUCCESS] TEST 7 - All 6 members match
[SUCCESS] TEST 8 - All 6 members match
[SUCCESS] TEST 9 - All 6 members match
[SUCCESS] TEST 10 - All 6 members match
[SUCCESS] TEST 11 - All 6 members match
[SUCCESS] TEST 12 - All 6 members match
[SUCCESS] TEST 13 - All 6 members match
[SUCCESS] TEST 14 - All 6 members match
[SUCCESS] TEST 15 - All 6 members match
[SUCCESS] TEST 16 - All 6 members match
[SUCCESS] TEST 17 - All 6 members match
```

Figure 2.5 Terminal output for update_complex_state functional test

3 example-sync-updated-2

Download example-sync-updated-2/ and review, build and run it.

a) Describe both the issues of deadlock and unbounded priority inversion and the root cause for both in the example code.

b) Fix the deadlock so that it does not occur by using a random back-off scheme to resolve. For the unbounded inversion, is there a real fix in Linux – if not, why not?

c) What about a patch for the Linux kernel? For example, Linux Kernel.org recommends the RT_PREEMPT Patch, also discussed by the Linux Foundation Realtime Start and this blog, but would this really help?

d) Read about the patch and describe why think it would or would not help with unbounded priority inversion. Based on inversion, does it make sense to simply switch to an RTOS and not use Linux at all for both HRT and SRT services?

3.1 Root Cause Analysis

3.1.1 deadlock and deadlock_timeout

The deadlock_timeout application experienced a segmentation fault because the macros, “THREAD_1” and “THREAD_2” were assigned to 1 and 2, respectively, but are later used as an index for the threadParams array which is zero indexed of size 2. Therefore threadParams[THREAD_2] is out of bounds and causes the application to segmentation fault, core dump, and crash. The fix was simply correcting these macros to 0 and 1, respectively to match the array indexes.

These applications demonstrate an unsafe deadlock scenario and then provide options to safely prevent the deadlock and an option to show a race condition. These applications do not demonstrate an uncontrolled priority inversion risk, as there are only two tasks. In the default unsafe mode, Thread 1 acquires lock A, sleeps, and then attempts to acquire lock B. However, before Thread 1 wakes up, Thread 2 acquires lock B, sleeps, and then attempts to acquire lock A. This is a classic example of deadlock as the threads acquire the locks in inverse order. The sleep() API call ensures that both threads are holding the lock the other thread wants after wake.

The deadlock application allows the application to hang forever after the deadlock. In contrast, the deadlock_timeout application uses the pthread_mutex_timedlock() API to prevent infinite hanging. In the case of deadlock, such as in the default unsafe mode, the timer will expire and return the code, “ETIMEDOUT”. This error code states that the lock was not able to be acquired in the allocated time, and allows the program to log the error and safely exit, as seen in Figure 3.1.

```

mason@jetson-orin-nano:~/RTES_5623/RTES-ECEE-5623-mcgaffin/example-sync-updated-2$ ./deadlock_timeout
Will set up unsafe deadlock scenario
Creating thread 0
Creating thread 1
Thread 1 started
THREAD 1 grabbing resource A @ 1782236523 sec and 367514404 nsec
Thread 1 GOT A
rsrcACnt=1, rsrcBCnt=0
will sleep
will try to join both CS threads unless they deadlock
Thread 2 started
THREAD 2 grabbing resource B @ 1782236523 sec and 367642311 nsec
Thread 2 GOT B
rsrcACnt=1, rsrcBCnt=1
will sleep
THREAD 1 got A, trying for B @ 1782236524 sec and 367671031 nsec
THREAD 2 got B, trying for A @ 1782236524 sec and 367828442 nsec
Thread 2 TIMEOUT ERROR
Thread 1 GOT B @ 1782236526 sec and 368137625 nsec with rc=0
rsrcACnt=1, rsrcBCnt=1
THREAD 1 got A and B
THREAD 1 done
Thread 1 joined to main
Thread 2 joined to main
All done

```

Figure 3.1 deadlock_timeout unsafe mode application output – TIMEOUT ERROR

Both applications offer a “safe” argument to the application execution command. When this argument is passed, the program prevents deadlock by running the threads sequentially. In safe mode, Thread 1 is allowed to complete before Thread 2 is created, as shown in Figure 3.2.

```

mason@jetson-orin-nano:~/RTES_5623/RTES-ECEE-5623-mcgaffin/example-sync-updated-2$ ./deadlock_timeout safe
Creating thread 0
Thread 1 started
THREAD 1 grabbing resource A @ 1782237260 sec and 917795819 nsec
Thread 1 GOT A
rsrcACnt=1, rsrcBCnt=0
will sleep
THREAD 1 got A, trying for B @ 1782237261 sec and 917913829 nsec
Thread 1 GOT B @ 1782237261 sec and 917967846 nsec with rc=0
rsrcACnt=1, rsrcBCnt=1
THREAD 1 got A and B
THREAD 1 done
Thread 1 joined to main
Creating thread 1
will try to join both CS threads unless they deadlock
Thread 2 started
THREAD 2 grabbing resource B @ 1782237261 sec and 918442834 nsec
Thread 2 GOT B
rsrcACnt=0, rsrcBCnt=1
will sleep
THREAD 2 got B, trying for A @ 1782237262 sec and 918622927 nsec
Thread 2 GOT A @ 1782237262 sec and 918766514 nsec with rc=0
rsrcACnt=1, rsrcBCnt=1
THREAD 2 got B and A
THREAD 2 done
Thread 2 joined to main
All done

```

Figure 3.2 deadlock_timeout safe mode application output – no errors

Both application also offer a “race” argument that eliminates the sleep() API call. This change makes the application indeterministic as seen in Figure 3.2. In the first execution, we get lucky, and Thread 1 completes before Thread 2 is started, similar to the safe execution. However, when running the application again, we see that Thread 2 acquires lock B before Thread 1 and we experience a deadlock, as reported by the timeout error.

```
mason@jetson-orin-nano:~/RTES_5623/RTES-ECEE-5623-mcgaffin/example-sync-updated-2$ ./deadlock_timeout race
Creating thread 0
Creating thread 1
Thread 1 started
THREAD 1 grabbing resource A @ 1782237112 sec and 239299297 nsec
Thread 1 GOT A
rsrcACnt=1, rsrcBCnt=0
THREAD 1 got A, trying for B @ 1782237112 sec and 239334114 nsec
will try to join both CS threads unless they deadlock
Thread 1 GOT B @ 1782237112 sec and 239345763 nsec with rc=0
rsrcACnt=1, rsrcBCnt=1
THREAD 1 got A and B
THREAD 1 done
Thread 2 started
THREAD 2 grabbing resource B @ 1782237112 sec and 239401828 nsec
Thread 2 GOT B
rsrcACnt=0, rsrcBCnt=1
THREAD 2 got B, trying for A @ 1782237112 sec and 239428964 nsec
Thread 2 GOT A @ 1782237112 sec and 239439461 nsec with rc=0
rsrcACnt=1, rsrcBCnt=1
THREAD 2 got B and A
THREAD 2 done
Thread 1 joined to main
Thread 2 joined to main
All done
mason@jetson-orin-nano:~/RTES_5623/RTES-ECEE-5623-mcgaffin/example-sync-updated-2$ ./deadlock_timeout race
Creating thread 0
Creating thread 1
Thread 1 started
Thread 2 started
THREAD 1 grabbing resource A @ 1782237126 sec and 389738064 nsec
Thread 1 GOT A
THREAD 2 grabbing resource B @ 1782237126 sec and 389738864 nsec
Thread 2 GOT B
will try to join both CS threads unless they deadlock
rsrcACnt=1, rsrcBCnt=0
THREAD 1 got A, trying for B @ 1782237126 sec and 390067224 nsec
rsrcACnt=1, rsrcBCnt=1
THREAD 2 got B, trying for A @ 1782237126 sec and 390081784 nsec
Thread 2 TIMEOUT ERROR
Thread 1 GOT B @ 1782237128 sec and 3900808997 nsec with rc=0
rsrcACnt=1, rsrcBCnt=1
THREAD 1 got A and B
THREAD 1 done
Thread 1 joined to main
Thread 2 joined to main
All done
```

Figure 3.3 deadlock_timeout race condition creates indeterministic execution

3.1.2 pthread3, pthread3amp, pthread3ok

These three applications demonstrate different bugs and wrong approaches to the priority inversion problem. The applications use an affinity set to restrict the application to a single core to simulate an AMP multi-service system with more than one service running on each core.

The first application, `pthread3`, is a basic bounded priority inversion demo. Without any changes, the application completes execution because the low priority service is established with non-real-time scheduling attributes, that causes the task to be run out of the RT FIFO scheduler hierarchy. This allows the task to be allocated some CPU time according to the round robin scheduling policy (`SCHED_RR`) assigned. This does not eliminate the priority inversion problem, but it does cause it to be bounded. The priority inversion is the classic sequence where the low priority task owns the mutex, is pre-empted by the high priority task. I noticed that when the low priority service is established with real-time scheduling attributes, the application enters an infinite loop as the start service has a higher priority than the low priority service. In order to simulate the similar bounded priority inversion, the start service calls `usleep(10)` to relinquish control of the CPU to the lower priority task for enough time to allow it to acquire the mutex and increment `CSent`. This allows the higher priority services to be created and simulate the bounded priority inversion once again.

The application, `pthread3amp`, has a critical bug that causes an infinite loop. The low priority service is initialized as a non-real-time thread which allows it to obtain CPU time under the `SCHED_RR` scheduling policy as mentioned before. However, the low priority service is assigned the callback, `simpleTask()`, which never enters the critical section to increment the `CSent` variable. Therefore, the starter service spinlocks forever, never starting the high or medium priority services. According the file header comment, “The intent for this code is to use priority amplification to break the unbounded inversion.” In order to fix this code, priority inheritance and priority ceiling emulation can be used. These changes are described in Section 3.2.

Finally, the application, `pthread3ok`, attempts to resolve the priority inversion problem by remove the blocking mechanism entirely. The application, based off `pthread3amp`, removes the `sharedMemSem` mutex that protects the critical section. By removing the blocking mechanism, the priority inversion is eliminated, but it introduces a different fault because the shared resource is no longer protected. Although this is not critical for this demonstration, in practice, this would be a critical bug that causes race conditions and indeterministic code.

3.2 Fixes & Kernel Patch

The deadlock code can be corrected, as seen in [this commit](#), by implementing a random back-off scheme at the cost of additional latency. This is a temporary solution to the problem that should really be solved by a more careful design. To implement the random back-off scheme, we use the `pthread_mutex_trylock()` API. If the thread, say thread 0, can obtain the lock, then it proceeds as normal. If not, the thread must release the resource it already holds, by unlocking the mutex, and yield the CPU for a random number of milliseconds. During the sleep, the other thread, thread 1, is able to obtain the lock and complete its execution. After the sleep, the thread 0 attempts to obtain both resources again, this time succeeding as the other thread has released both resources. See Figure 3.4 below

```

mason@jetson-orin-nano:~/RTES_5623/RTES-ECEE-5623-mcgaiffin/example-sync-updated-2$ ./deadlock unsafe
Will set up random backoff scenario
Creating thread 0
Thread 0 spawned
Creating thread 1
Thread 0 grabbing resources
Thread 1 spawned
rsrcACnt=1, rsrcBCnt=0
will try to join CS threads unless they deadlock
Thread 1 grabbing resources
Thread 0 got A, trying for B
Thread 0 failed to get B. Backing off...
Thread 1 got B, trying for A
Thread 1 got B and A
Thread 1 done
Thread 0 grabbing resources
Thread 0 got A, trying for B
Thread 0 got A and B
Thread 0 done
Thread 0: 882bc840 done
Thread 1: 87aac840 done
All done

```

Figure 3.4 Random back-off scheme to fix deadlock

The unbounded priority inversion can be solved with standard Linux if the synchronization mechanism are configured correctly. The kernel does not know which resources are shared or which tasks should inherit priority, so it is up to the developer to be aware of these potential issues. However, these solutions introduce increased latency that make them unsuitable for hard real-time applications. The PREEMPT_RT patch is focused on removing unbounded latency in the kernel. Rostedt [11] claims that the patch gives you something “very close” to hard real-time similar to what an RTOS can do. That said, the patch is not suited for all hard real-time deadlines, especially if loss of life could be an outcome of a missed deadline. For our example, applying this patch is not going to fix anything. The priority inversion is created strictly in user space. However, using this patch OS would decrease the scheduling latency in the solution described below and make the execution time much more deterministic.

The file, pthread3amp, attempts to solve the priority inversion but lacks a complete solution. The following changes are made as seen in [this commit](#). The first major change was to spawn the low priority service with the cirticalSectionTask as opposed to the simpleTask. This task, and the starter service were made real-time services, eliminating NRT services. The starter service was configured with the highest priority so that all tasks were created before the priority inversion case to accurately demonstrate priority amplification. The usleep() API is used by the starter service to allow the low priority service to enter the critical section before spawning the high and medium priority services. This was the primary bug causing the spin lock. The next change to allow proper priority amplification was to correctly initialize the mutex attributes before initializing the mutex. Finally, changes to clean up unused variable, the CPU affinity logic, the scheduling policy logic, and reworking some of the application output. The result, as shown in files ‘[inherit.txt](#)’ and ‘[protect.txt](#)’, demonstrate the priority amplification to mitigate unbounded priority inversion using both PTHREAD_PRIO_INHERIT and PTHREAD_PRIO_PROTECT, respectively.

4 POSIX Message Queues

Review POSIX-Examples-Updated and POSIX_MQ_LOOP and build the code related to POSIX message queues and run them to learn about basic use.

a) First, re-write the simple message queue demonstration code so that it uses RT-Linux Pthreads (FIFO) instead of SCHED_OTHER, and then write a brief paragraph describing how the use of a heap message queue and a simple message queue for applications are similar and how they are different.

b) Message queues are often suggested as a better way to avoid MUTEX priority inversion. Would you agree that this really circumvents the problem of unbounded inversion? If so why, if not, why not?

4.1 Implementing RT-Linux Pthreads – FIFO

First, the single message posix_mq application was re-written to use posix threads in [this commit](#). The application output is shown in Figure 4.1. This example uses a POSIX simple message queue at '/dev/mqueue/send_receive_mq'. The data transfer is handled by the kernel. The caller uses the syscall "mq_open" to open the simple message queue from both the sender and receiver and the "mq_send" and "mq_receive" syscalls to enqueue and dequeue messages, respectively.

```
mason@jetson-orin-nano:~/RTES_5623/RTES-ECEE-5623-mcgaffin/POSIX-Examples-Updated$ sudo ./posix_mq
Sender Thread Created
sender opened mq
Receiver Thread Created
receiver opened mq
send: message successfully sent
receive: msg This is a test, and only a test, in the event of real emergency, you would be instructed.... received with priority = 30, length = 93
Sender joined
Receiver joined
```

Figure 4.1 posix_mq using Posix threads

The looped posix_mq application works very similarly to the single message application mentioned above. I made [these changes](#) to get the source code to compile and run. The only difference is that the sender and receiver threads run forever, sending and receiving messages, respectively, until an error occurs. As seen in Figure 4.2 below, the sender thread is created first, immediately sending messages. Then, the receiver thread is created and begins dequeuing messages from the queue. As seen in Figure 4.3, the sender and receiver work asynchronously. In order to get a better understanding of what was happening in the message queue mechanism, I used the watch command as described below, to see the queue size fluctuate. This tool, however, is very limited by its update rate, only 0.1s at best, while messages are being transferred thousands of times per second. In figure 4.4, an instance of the watch is captured, showing the change in QSIZE as indicated by the highlighted text, prompted by passing the '-d' argument.

```
mason@jetson-orin-nano:~/RTES_5623/RTES-ECEE-5623-mcgaffin/POSIX_MQ_LOOP$ sudo ./posix_mq | head -n 100
Sender Thread Created with rc=0
sender - thread entry
sender - sending message of size=93
send: message successfully sent, rc=0
sender - sending message of size=93
send: message successfully sent, rc=0
sender - sending message of size=93
send: message successfully sent, rc=0
sender - sending message of size=93
send: message successfully sent, rc=0
sender - sending message of size=93
send: message successfully sent, rc=0
```

Figure 4.2 posix_mq loop application and sender thread start

```
Receiver Thread Created with rc=0
pthread join send
receiver - thread entry
receiver - awaiting message
receive: msg This is a test, and only a test, in the event of real emergency, you would be instructed....
received with priority = 30, length = 93
receiver - awaiting message
send: message successfully sent, rc=0
sender - sending message of size=93
receive: msg This is a test, and only a test, in the event of real emergency, you would be instructed....
received with priority = 30, length = 93
receiver - awaiting message
send: message successfully sent, rc=0
sender - sending message of size=93
receive: msg This is a test, and only a test, in the event of real emergency, you would be instructed....
received with priority = 30, length = 93
receiver - awaiting message
send: message successfully sent, rc=0
sender - sending message of size=93
receive: msg This is a test, and only a test, in the event of real emergency, you would be instructed....
received with priority = 30, length = 93
receiver - awaiting message
```

Figure 4.3 posix_mq loop receiver thread start

COMMAND: `sudo watch -d -p -n 0.1 cat /dev/mqueue/send_receive_mq`

```
Every 0.1s: cat /dev/mqueue/send_receive_mq jetson-orin-nano: Tue Jun 23 20:36:17 2026
QSIZE:465 NOTIFY:0 SIGNO:0 NOTIFY_PID:0
```

Figure 4.4 watch command live output capture

4.2 Heap vs Simple Message Queues

A heap message queue and simple message queue have the same goal: to enable asynchronous data transfer between threads/processes. Both mechanisms use a first-in first out (FIFO) data structure, that enables a sender to send messages in order to a receiver. The differences arise when you consider data and memory management. Simple message queues copy the payload,

along with the payload length, and priority, to a kernel-space buffer that the receiver can copy from. The isolation of user and kernel space make the transaction very secure but introduce notable latency due to the copy actions. In contrast, the heap message queue allocates a block of memory on the shared heap, writes to it, and only the pointer is passed into the queue making it significantly lighter weight and eliminating the data copying. The increased speed comes at a risk though, as developers are responsible for avoiding memory leaks and race conditions.

I was able to create a heap message queue in [this commit](#). The application creates a shared resource, the message queue, with an embedded mutex lock. The structure uses linked list nodes to track the messages incoming and exiting the queue. The sender allocates memory for the new node and its message, and inserts it at the tail of the list (queue). The receiver reads messages from the head of the queue and then frees the memory of the node, updating the head to the next message available. The application also tracks when the threads are created and joined. Using this and the number of messages sent, we can calculate an approximate data rate. As seen in Figure 4.5, below, the Sender thread is created first and starts sending messages right away. Once the receiver thread is up and running, it will begin to asynchronously dequeue messages. It is difficult to tell with the printf output due to stdout serialization locks. The output in Figure 4.5 shows semi-expected asynchronous sending and receiving, but often times, the serial output will look more batched due to these locks.

```
mason@jetson-orin-nano:~/RTES_5623/RTES-ECEE-5623-mcgaffin/POSIX_MQ_LOOP$ sudo ./heap_mq |{ head -n 8; echo; tail -n 20; }
Creating threads at 1782319428.00119
Sender Thread Created
Receiver Thread Created
[CPU 4] SENT message: TEST MESSAGE 0 | Count=1
[CPU 4] SENT message: TEST MESSAGE 1 | Count=2
[CPU 4] SENT message: TEST MESSAGE 2 | Count=3
[CPU 4] SENT message: TEST MESSAGE 3 | Count=4
[CPU 4] SENT message: TEST MESSAGE 4 | Count=5

[CPU 3] RECEIVED message: TEST MESSAGE 993 | Count=0
[CPU 4] SENT message: TEST MESSAGE 994 | Count=1
[CPU 3] RECEIVED message: TEST MESSAGE 994 | Count=0
[CPU 4] SENT message: TEST MESSAGE 995 | Count=1
[CPU 4] SENT message: TEST MESSAGE 996 | Count=2
[CPU 4] SENT message: TEST MESSAGE 997 | Count=3
[CPU 3] RECEIVED message: TEST MESSAGE 995 | Count=2
[CPU 4] SENT message: TEST MESSAGE 998 | Count=3
[CPU 3] RECEIVED message: TEST MESSAGE 996 | Count=2
[CPU 4] SENT message: TEST MESSAGE 999 | Count=3
[CPU 3] RECEIVED message: TEST MESSAGE 997 | Count=2
[CPU 3] RECEIVED message: TEST MESSAGE 998 | Count=1
[CPU 4] SENT message: TEST MESSAGE 1000 | Count=2
[CPU 3] RECEIVED message: TEST MESSAGE 999 | Count=1
[CPU 3] RECEIVED message: TEST MESSAGE 1000 | Count=0
Sender joined
Receiver joined
Joining threads at 1782319428.01371
Elapsed time = 0.01252
Data rate = 79894.54836 messages per second
```

Figure 4.5 First 8 lines and last 20 lines of heap_mq application output

A big concern with heap message queues is data leaks. So, we use Valgrind to verify that all of the dynamically allocated memory is properly deallocated and freed. As seen in Figure 4.6, no leaks are possible and there are no errors.

COMMAND:

```
sudo valgrind --leak-check=full ./heap_mq
```

```
==3593550==
==3593550== HEAP SUMMARY:
==3593550==      in use at exit: 0 bytes in 0 blocks
==3593550==    total heap usage: 440,455 allocs, 440,455 frees, 10,572,416 bytes allocated
==3593550==
==3593550== All heap blocks were freed -- no leaks are possible
==3593550==
==3593550== For lists of detected and suppressed errors, rerun with: -s
==3593550== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Figure 4.6 heap_mq Valgrind testing output

4.3 Unbounded Inversion

Implementing a POSIX message queue does not magically solve unbounded priority inversion created by mutexes. It is a different mechanism entirely, used for different cases. Mutexes should be prioritized for synchronous execution and data transfer whereas message queues should be used in asynchronous cases.

The message queue mechanism allows messages to have priorities. If a thread sends a message with a certain priority, it will automatically jump ahead of all messages in the queue with a lower priority. This has parallels to priority inheritance, but it does not affect when threads are scheduled, thus introducing the risk of unbounded priority inversion in synchronous cases. Message queues are designed to create asynchronous execution. A high-priority task does not need to wait for a reply from a lower priority task and can continue to be unblocked. However, if a low priority task attempts to send a message to a full queue, it will for space to open, causing priority inversion as the higher priority tasks are now blocked.

In conclusion, Mutexes and message queues are distinct mechanisms with unique use cases. When designing a system, architects/engineers/developers need to be aware of the tradeoffs that each mechanism presents to the specific situation.

5 Linux Watchdog Daemon

5.1 Using Watchdog in Complex State Update

A watchdog timer is especially useful in the case of indefinite deadlock as it can return the system to a known state if not serviced within the allocated time period. The Linux watchdog daemon can do this in hardware or software, depending on system specifications. The watchdog can be configured to reset the kernel, notify users, and perform other system housekeeping when the watchdog is not serviced within the allocated period. This can be extremely useful for a system that enters an indefinite deadlock as the system can be reset to a known state and quickly restart. The watchdog daemon requires careful configuration depending on the user/system needs.

To explore watchdogs, we modified the source code from [Section 2.3](#) in [this commit](#), to use the `pthread_mutex_timedlock()` API. In order to properly demonstrate this concept, the update rate was slowed to 0.2Hz (once every five (5) seconds). The reader thread has been changed to wait for new data. If data becomes available, it is immediately logged. If no updates after four (4) seconds, the reader reports such in syslog, as seen in Figure 5.1. As seen in the output by the built-in syslog timestamps and the double precision timestamps, the reader thread reports if 4 seconds have passed since the last writer update. If the writer makes an update to the state, it is immediately followed by the reader log.

```
Jun 24 12:38:32 jetson-orin-nano complex_state_watchdog 36369777: starting complex state update - Duration: 60, Update rate: 0.20, Reader timeout: 4s
Jun 24 12:38:36 jetson-orin-nano complex_state_watchdog 36369777: [1782326316.8446] READER (CPU 3) - No new data available
Jun 24 12:38:37 jetson-orin-nano complex_state_watchdog 36369777: [1782326317.8451] WRITER (CPU 4) - lat=0.050007 lon=1.000149 alt=1.250187 roll=-0.958712 pitch=0.992164 yaw=0.284378
Jun 24 12:38:37 jetson-orin-nano complex_state_watchdog 36369777: [1782326317.8456] READER (CPU 3) - lat=0.050007 lon=1.000149 alt=1.250187 roll=-0.958712 pitch=0.992164 yaw=0.284378
Jun 24 12:38:41 jetson-orin-nano complex_state_watchdog 36369777: [1782326321.8463] READER (CPU 3) - No new data available
Jun 24 12:38:42 jetson-orin-nano complex_state_watchdog 36369777: [1782326322.8463] WRITER (CPU 4) - lat=0.100020 lon=2.000398 alt=2.500497 roll=-0.545688 pitch=0.881769 yaw=-0.837988
Jun 24 12:38:42 jetson-orin-nano complex_state_watchdog 36369777: [1782326322.8466] READER (CPU 3) - lat=0.100020 lon=2.000398 alt=2.500497 roll=-0.545688 pitch=0.881769 yaw=-0.837988
Jun 24 12:38:46 jetson-orin-nano complex_state_watchdog 36369777: [1782326326.8473] READER (CPU 3) - No new data available
Jun 24 12:38:47 jetson-orin-nano complex_state_watchdog 36369777: [1782326327.8477] WRITER (CPU 4) - lat=0.150034 lon=3.000674 alt=3.750843 roll=0.647723 pitch=0.459353 yaw=-0.761876
Jun 24 12:38:47 jetson-orin-nano complex_state_watchdog 36369777: [1782326327.8480] READER (CPU 3) - lat=0.150034 lon=3.000674 alt=3.750843 roll=0.647723 pitch=0.459353 yaw=-0.761876
Jun 24 12:38:51 jetson-orin-nano complex_state_watchdog 36369777: [1782326331.8487] READER (CPU 3) - No new data available
Jun 24 12:38:52 jetson-orin-nano complex_state_watchdog 36369777: [1782326332.8491] WRITER (CPU 4) - lat=0.200047 lon=4.000948 alt=5.001185 roll=0.914869 pitch=-0.355492 yaw=0.403750
Jun 24 12:38:52 jetson-orin-nano complex_state_watchdog 36369777: [1782326332.8494] READER (CPU 3) - lat=0.200047 lon=4.000948 alt=5.001185 roll=0.914869 pitch=-0.355492 yaw=0.403750
Jun 24 12:38:56 jetson-orin-nano complex_state_watchdog 36369777: [1782326336.8500] READER (CPU 3) - No new data available
Jun 24 12:38:57 jetson-orin-nano complex_state_watchdog 36369777: [1782326337.8505] WRITER (CPU 4) - lat=0.250061 lon=5.001228 alt=6.251535 roll=-0.126264 pitch=-0.991551 yaw=0.991997
Jun 24 12:38:57 jetson-orin-nano complex_state_watchdog 36369777: [1782326337.8508] READER (CPU 3) - lat=0.250061 lon=5.001228 alt=6.251535 roll=-0.126264 pitch=-0.991551 yaw=0.991997
Jun 24 12:39:01 jetson-orin-nano complex_state_watchdog 36369777: [1782326341.8515] READER (CPU 3) - No new data available
Jun 24 12:39:02 jetson-orin-nano complex_state_watchdog 36369777: [1782326342.8515] WRITER (CPU 4) - lat=0.300072 lon=6.001433 alt=7.501792 roll=-0.986901 pitch=-0.355771 yaw=0.161328
Jun 24 12:39:02 jetson-orin-nano complex_state_watchdog 36369777: [1782326342.8518] READER (CPU 3) - lat=0.300072 lon=6.001433 alt=7.501792 roll=-0.986901 pitch=-0.355771 yaw=0.161328
Jun 24 12:39:06 jetson-orin-nano complex_state_watchdog 36369777: [1782326346.8525] READER (CPU 3) - No new data available
```

Figure 5.1 syslog output for complex_state_watchdog

6 References

- [1] J. Pratt and S. Siewert, Real-Time Embedded Components and Systems with Linux and RTOS, 2nd ed. Herndon, VA, USA: Mercury Learning and Information, 2016.
- [2] "Linux man pages," Die.net. Accessed: Jun. 5, 2026. [Online]. Available: <https://linux.die.net/man/>
- [3] M. Kerrisk, "Linux man-pages online," Man7.org. Accessed: Jun. 5, 2026. [Online]. Available: <http://man7.org/linux/man-pages/index.html>
- [4] L. Sha, R. Rajkumar, and J. P. Lehoczky, "Priority inheritance protocols: an approach to real-time synchronization," *IEEE Trans. Comput.*, vol. 39, no. 9, pp. 1175-1185, Sep. 1990, doi: 10.1109/12.57058.
- [5] S. Siewert, "SoC drawer: Shared resource management," *Real-Time Embedded Systems (RTES) Course Materials*, University of Colorado Boulder. [Online].
- [6] Jonathan Corbet, "Re: [PATCH 1/19] MUTEX: Introduce simple mutex implementation," *Linux Weekly News (LWN)*, Mar. 2, 2006. [Online]. Available: <https://lwn.net/Articles/178258/>
- [7] Ingo Molnar, "PI-futex: -V1," *Linux Weekly News (LWN)*, Mar. 17, 2006. [Online]. Available: <https://lwn.net/Articles/177111/>
- [8] Jonathan Corbet, "Priority inheritance in the kernel," *Linux Weekly News (LWN)*, Apr. 4, 2006. [Online]. Available: <https://lwn.net/Articles/178253/>
- [9] Red Hat, Inc., "Understanding RHEL for Real Time," *Red Hat Enterprise Linux for Real Time 8 Documentation*, 2024. [Online]. Available: https://docs.redhat.com/en/documentation/red_hat_enterprise_linux_for_real_time/8/html-single/understanding_rhel_for_real_time/index
- [10] Real-Time Linux Project, "Real-Time Linux Wiki," *Linux Foundation Wiki*, Jan. 2026. [Online]. Available: <https://wiki.linuxfoundation.org/realtime/start>
- [11] S. Rostedt, "Intro to Real-Time Linux for Embedded Developers," *The Linux Foundation Blog*, Mar. 21, 2013. [Online]. Available: <https://www.linuxfoundation.org/blog/blog/intro-to-real-time-linux-for-embedded-developers>
- [12] Crawford Space Consulting, "Linux Watchdog Daemon - Configuring," *PSC's Web Resources*, Jan. 2011. [Online]. Available: https://www.crawford-space.co.uk/old_psc/watchdog/watchdog-configure.html

7 Appendix

Github repository: https://github.com/mamc3334/RTES_5623

Pull request: https://github.com/mamc3334/RTES_5623/pull/3

Forked RTES Github repository: <https://github.com/mamc3334/RTES-ECEE-5623-mcgaffin>

Pull request: <https://github.com/mamc3334/RTES-ECEE-5623-mcgaffin/pull/3>